

Лабораторная работа № 7.

Развёртывание Node.js REST API на PaaS Render

Тема: Публикация Internet Shop API на Render: Git, environment variables, PostgreSQL, migrations и public URL

Цель: Развернуть Internet Shop REST API на Render, подключить managed PostgreSQL и проверить работу публичного HTTPS API.

Дисциплина: Разработка браузерных приложений

Лабораторная работа №7. Развёртывание Node.js REST API на PaaS Render

В лабораторной работе необходимо опубликовать Internet Shop REST API на платформе Render. Проект будет получать исходный код из GitHub, использовать отдельную Render PostgreSQL database, environment variables, Prisma migrations и публичный HTTPS URL.

1. Ожидаемый результат

- Проект Internet Shop находится в GitHub repository.
- Создана Render PostgreSQL database.
- Создан Render Web Service для Node.js.
- API использует process.env.PORT и host 0.0.0.0.
- DATABASE_URL хранится в Render Environment.
- Prisma Client генерируется во время build.
- Prisma migrations применяются при запуске deployment.
- REST API доступен по публичному HTTPS URL.
- Health endpoint /health/ready возвращает успешный результат.
- Студент умеет открыть deployment logs и найти причину ошибки.

2. Предварительная проверка проекта

Версия Node.js

```
node --version  
npm --version
```

В учебном проекте используется Node.js 22. Перед публикацией убедитесь, что зависимости и тесты работают локально.

Чистая установка и тесты

```
npm ci  
npx prisma generate  
npm test
```

Локальный запуск

```
npm start
```

После запуска проверьте `/health/live`, `/health/ready` и один REST endpoint, например `GET /api/products`.

3. Подготовка `server.js` к cloud deployment

Render передаёт приложению номер HTTP-порта через environment variable `PORT`. Сервер должен слушать интерфейс `0.0.0.0`.

Пример `server.js`

```
import app from './app.js';

const PORT = Number(process.env.PORT ?? 3000);
const HOST = '0.0.0.0';

const server = app.listen(PORT, HOST, () => {
  console.log(`API started on ${HOST}:${PORT}`);
});
```

Не фиксируйте production port значением 3000. Число 3000 используется только как fallback для локальной разработки.

4. Проверка `scripts` в `package.json`

Scripts для deployment

```
{
  "scripts": {
    "start": "node server.js",
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --runInBand",
    "prisma:generate": "prisma generate",
    "db:deploy": "prisma migrate deploy"
  },
  "engines": {
    "node": "22.x"
  }
}
```

Если в вашем проекте entry point имеет другое имя или путь, измените script `start` соответственно.

5. Проверка `production readiness`

- `.env` находится в `.gitignore`.
- `package-lock.json` добавлен в Git.
- `prisma/schema.prisma` добавлен в Git.
- Каталог `prisma/migrations` добавлен в Git.
- `JWT_SECRET` не записан в исходном коде.
- Health endpoints не возвращают secrets и stack trace.
- Приложение корректно завершает startup при отсутствии обязательных variables.

6. Отправка актуальной версии в GitHub

Commit и push

```
git status
git add .
git commit -m "Prepare API for Render deployment"
git push
```

Убедитесь, что последняя версия проекта видна в GitHub repository и CI из предыдущей лабораторной завершается успешно.

7. Создание PostgreSQL в Render

1. Откройте Render Dashboard.
2. Создайте новый PostgreSQL resource.
3. Задайте понятное имя, например internet-shop-db.
4. Выберите доступный учебный plan.
5. Выберите регион и запомните его — Web Service создаём в том же регионе.
6. Дождитесь состояния, при котором database готова к подключениям.

На странице database найдите параметры подключения. Для Web Service в том же регионе используйте Internal Database URL.

8. Почему используем Internal Database URL

- Соединение предназначено для Render services в том же регионе.
- Трафик идёт по private network Render.
- Не требуется обращаться к database через public internet.
- Этот URL будет значением DATABASE_URL Web Service.

9. Создание Web Service

7. В Render Dashboard выберите New Web Service.
8. Подключите GitHub account, если он ещё не подключён.
9. Выберите repository Internet Shop.
10. Выберите branch main.
11. Language: Node.
12. Выберите тот же регион, что и PostgreSQL.
13. Задайте имя сервиса, например internet-shop-api.

10. Build Command

Build Command Render

```
npm ci && npx prisma generate
```

npm ci устанавливает зависимости по package-lock.json. Затем prisma generate создаёт Prisma Client для текущего deployment.

11. Start Command

Start Command для учебного проекта

```
npm run db:deploy && npm start
```

Сначала prisma migrate deploy применяет pending migrations. После успешной migration запускается Node.js REST API. В более сложной production-среде migrations обычно выносят в отдельный pre-deploy этап.

12. Добавление Environment Variables

Переменные Render

Key	Value
NODE_ENV	production
DATABASE_URL	Internal Database URL созданной Render PostgreSQL
JWT_SECRET	Отдельное случайное production значение
CORS_ORIGIN	URL frontend, только если используется ограничение CORS

Не копируйте production secrets в исходный код, README или GitHub repository.

13. Создание JWT_SECRET

Для учебного проекта можно сгенерировать случайный секрет локально и сохранить только в Render Environment.

Пример генерации случайного значения через Node.js

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Скопируйте результат в JWT_SECRET. Не добавляйте его в отчёт полностью: на скриншоте значение должно быть скрыто.

14. Настройка Health Check

В настройках Web Service задайте HTTP Health Check Path: /health/ready.

Пример readiness endpoint

```
app.get('/health/ready', async (req, res) => {
  try {
    await prisma.$queryRaw`SELECT 1`;
    res.status(200).json({ status: 'ready' });
  } catch (error) {
    res.status(503).json({ status: 'not_ready' });
  }
});
```

Readiness endpoint проверяет, что Express работает и PostgreSQL доступна.

15. Первый deployment

14. Сохраните настройки Web Service.
15. Render начнёт build.
16. Откройте Deploys и следите за текущим deployment.
17. В logs найдите npm ci.
18. Проверьте успешный prisma generate.
19. Проверьте prisma migrate deploy.
20. Проверьте сообщение запуска Node.js.
21. Дождитесь успешного health check.

16. Проверка публичного URL

После успешного deployment Render выдаёт HTTPS URL вида <https://имя-сервиса.onrender.com>. Используйте фактический URL вашего сервиса.

Примеры запросов

```
GET https://<your-service>.onrender.com/health/live
GET https://<your-service>.onrender.com/health/ready
GET https://<your-service>.onrender.com/api/products
```

Не используйте localhost:3000 при проверке Render — это адрес локального компьютера, а не опубликованного сервиса.

17. Проверка API через PowerShell

Health check

```
Invoke-RestMethod -Method Get `
  -Uri "https://<your-service>.onrender.com/health/ready"
```

Получение товаров

```
Invoke-RestMethod -Method Get `
  -Uri "https://<your-service>.onrender.com/api/products"
```

18. Проверка authentication

Отправьте существующий запрос login к публичному API. Полученный JWT используйте для protected endpoint. Формат endpoint и body должен соответствовать вашему Internet Shop проекту.

Логика проверки

```
POST /api/auth/login
  ↓
access token
  ↓
Authorization: Bearer <token>
  ↓
protected endpoint
```

19. Просмотр logs

22. Откройте Web Service.
23. Перейдите в Logs.
24. Найдите сообщение запуска приложения.
25. Выполните GET /api/products.
26. Найдите соответствующий application log, если logging настроен.
27. При ошибке определите, относится она к build, migration, startup или runtime.

20. Эксперимент с auto-deploy

Внесите безопасное изменение, например измените значение version в health response или добавьте информационный log. Затем выполните commit и push.

Новая версия

```
git add .
git commit -m "Update API version marker"
```

```
git push
```

Откройте Render Deploys и убедитесь, что новая версия была собрана и опубликована автоматически. Если настроен режим After CI Checks Pass, deployment начнётся после успешного завершения GitHub Actions.

21. Типичные ошибки

Диагностика Render

Проблема	Что проверить
No open ports detected	process.env.PORT и bind на 0.0.0.0
DATABASE_URL missing	Render Environment Variables
Prisma connection error	Internal Database URL и одинаковый region
Migration failed	prisma/migrations и структура schema
Application exited	Start Command и startup logs
401/403	JWT_SECRET, token и Authorization header
CORS blocked	CORS_ORIGIN и CORS middleware

22. Контрольные вопросы

28. Что такое deployment?
29. Чем PaaS отличается от VPS?
30. Какие части инфраструктуры Render берёт на себя?
31. Почему Node.js приложение должно использовать process.env.PORT?
32. Для чего нужен host 0.0.0.0?
33. Почему .env не загружают в repository?
34. Чем Internal Database URL отличается от External Database URL?
35. Для чего используется prisma migrate deploy?
36. Что проверяет /health/ready?
37. Где искать причину failed deployment?
38. Что происходит после push при включённом auto-deploy?

23. Требования к отчёту

39. Цель лабораторной работы.
40. Фрагмент server.js с PORT и 0.0.0.0.
41. Фрагмент package.json со scripts и Node.js version.
42. Скриншот созданной PostgreSQL без отображения secrets.
43. Скриншот настроек Web Service.
44. Build Command и Start Command.
45. Список names environment variables без раскрытия secret values.
46. Скриншот успешного deployment.
47. Результат /health/ready.
48. Результат GET /api/products.
49. Фрагмент Render logs.
50. Скриншот повторного auto-deploy после нового push.
51. Ответы на контрольные вопросы.

24. Критерии выполнения

Критерии

Критерий	Результат
API опубликован на Render	Обязательно
Используется Render PostgreSQL	Обязательно
DATABASE_URL находится в Environment	Обязательно
Prisma Client генерируется при build	Обязательно
Migrations применяются	Обязательно
Health check работает	Обязательно
Публичный REST endpoint отвечает	Обязательно
Logs доступны и проанализированы	Обязательно
Продемонстрирован повторный deployment после push	Обязательно

25. Вывод

В результате лабораторной работы Internet Shop REST API опубликован на PaaS Render. Исходный код поступает из GitHub, Render выполняет build и запуск приложения, PostgreSQL предоставляется отдельным managed resource, secrets хранятся в Environment, Prisma поддерживает структуру database через migrations, а работоспособность контролируется health checks и logs.